

CS383 - Software Engineering

Software Testing (I)

Outline

- Software testing
- Verification vs. validation
- Software inspection and testing
- Testing levels
- Testing strategies

Program testing

- An action that is intended to show whether the program is developed correctly.
 - Program under test is executed against input data
 - The results are checked if they are correct as they should be
 - The goal is to reveal any hidden errors

Program testing goals

- Test what?
 - Ideally, all features of program must be tested
 - Different processes that contribute to a functionality also must be internally tested
- To discover situations in which the behavior of the software is incorrect, undesirable or does not conform to its specification.
- Defect testing is concerned with rooting out undesirable system behavior
 - Eg, system crashes, unwanted interactions with other systems, incorrect computations and data corruption.

Software testing and confidence

Aim of testing is to establish confidence that the system is ‘fit for purpose’:

- **Software purpose:** the level of confidence depends on how critical the software is to an organisation
- **User expectations:** Users may have low expectations of certain kinds of software
- **Marketing environment:** getting a product to market early may be more important than finding defects in the program

Verification vs. validation

- Verification: “Are we building the product right?”
 - Discovering defects/bugs and errors in product
 - Use oracles (expected outputs) to do the testing
- Validation: “Are we building the right product?”
 - Checking the product against user/client requirements
 - Use prototyping for extracting exact behaviour of features

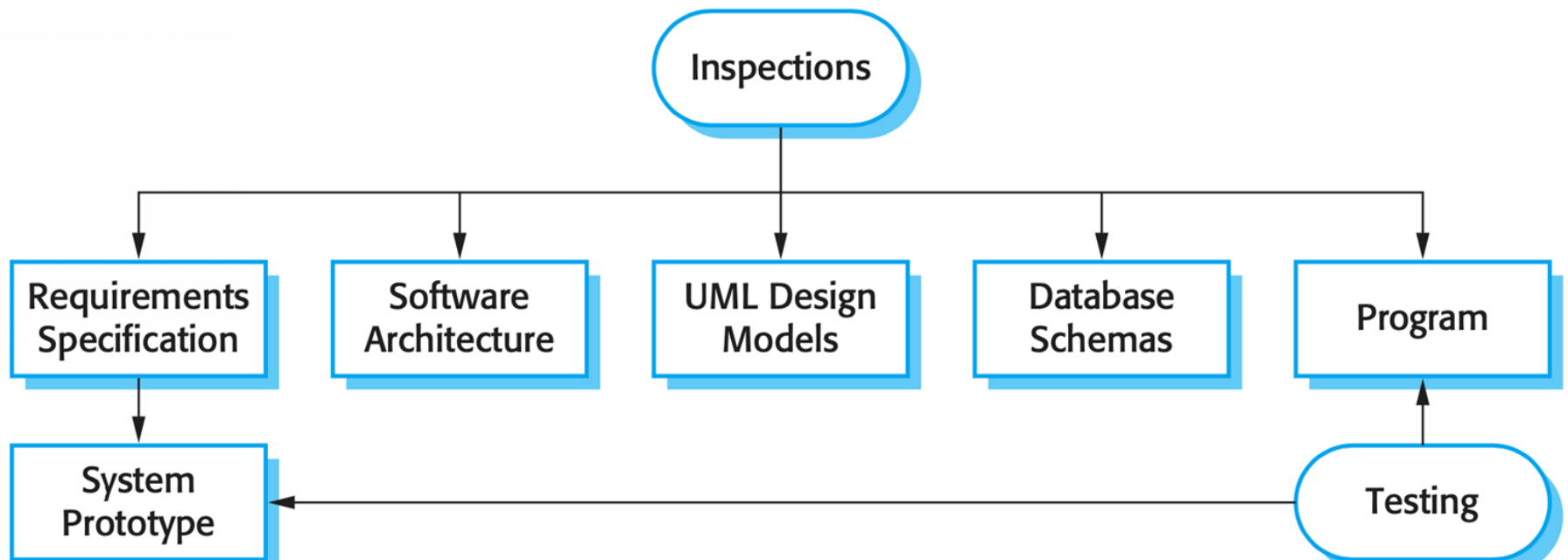


Software verification (cont.)

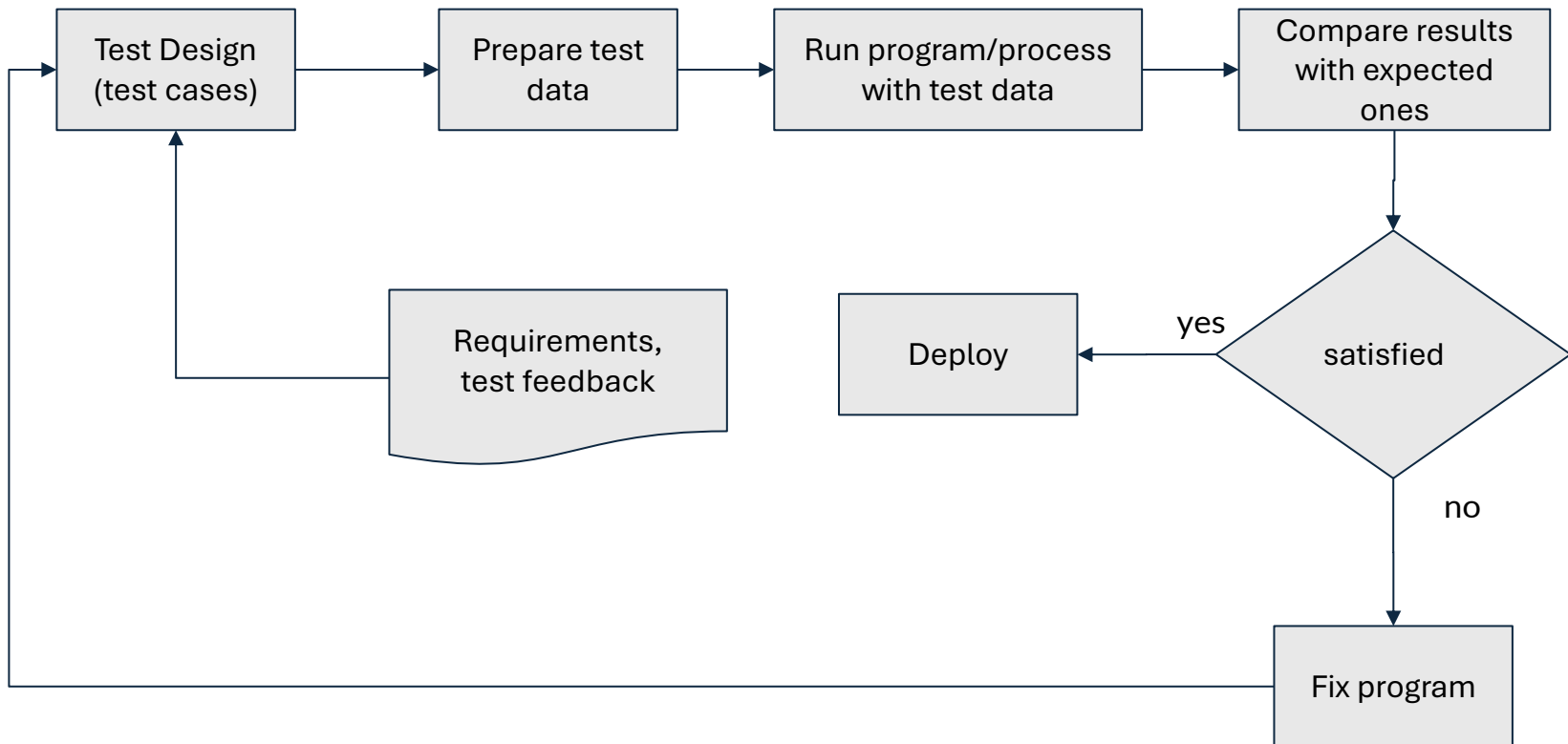
- Software inspections (static verification)
 - Concerned with analysis of the static system representation
 - May be supplemented by tool-based document and code analysis.
 - Check any readable software, requirements, models etc
 - Basically, use knowledge of of the system, its domain, modeling language to discover errors
- Software testing (dynamic verification)
 - Concerned with exercising and observing product behaviour
 - The system is executed with test data and its operational behaviour is observed.

Software verification: inspections and testing

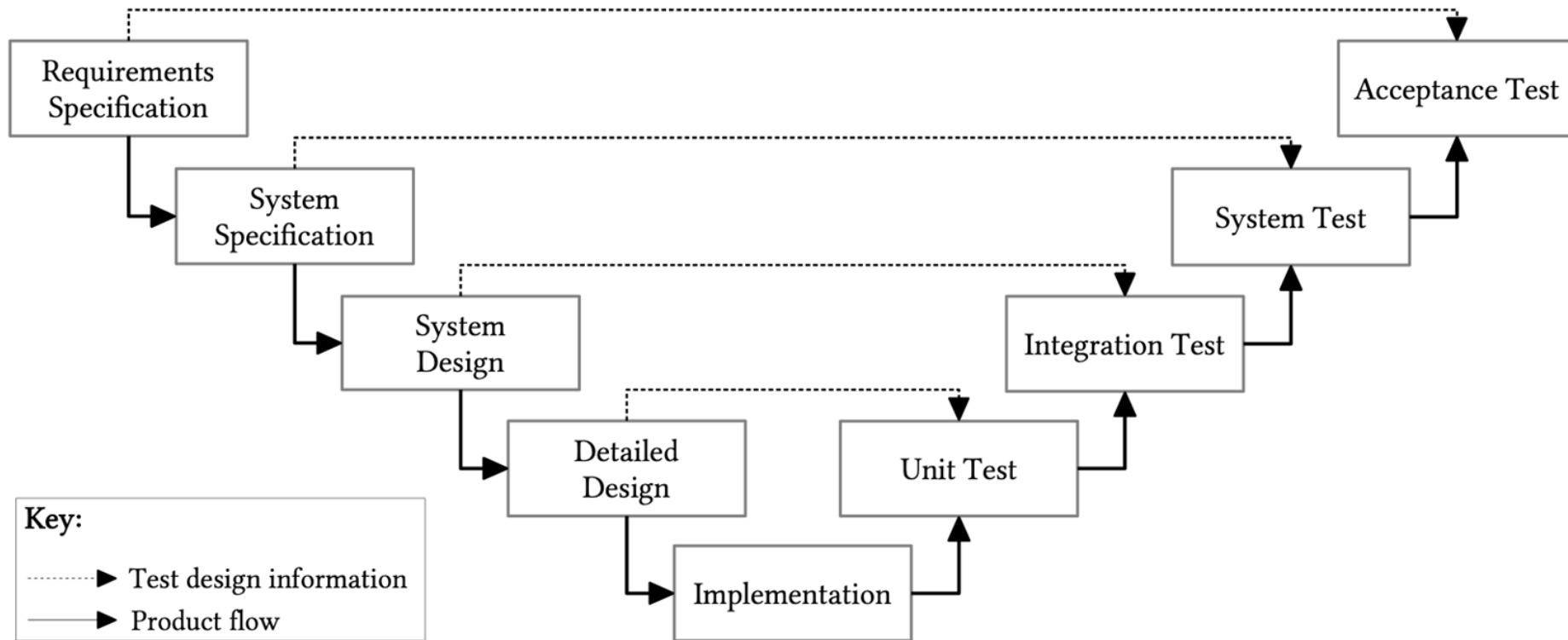
- The figure shows states that can be inspected for errors and those tested for errors.



Software testing process model



Software testing levels (V model)



Unit test, integration (component) test and system test are known as development tests

Development tests

- Unit testing (class - method level)
 - Individual program units/processes or object classes are tested
 - Unit testing should focus on testing the functionality of objects or methods
- Component testing (package level)
 - Several individual units are integrated to create composite components
 - Component testing should focus on testing component interfaces
- System testing (all packages)
 - System components are integrated and the system is tested as a whole
 - System testing should focus on testing component interactions

Automated testing

- Whenever possible, unit testing should be automated
 - Tests are run and checked without manual intervention
 - There are automation tools
 - Eg. JUnit or cppunit
- Unit testing frameworks provide generic test classes that you extend to create specific test cases
 - Frameworks can then run all of the tests that you have implemented

Automated testing (cont.)

- A setup part
 - Initialize the system with the test cases
- A call part
 - call the object or method to be tested
- An assertion and report part
 - compare the result of the call with the expected result. If the assertion evaluates to true, the test has been successful if false, then it has failed

Testing strategies

- Partition testing into packages, processes, methods, classes and folders
- For acceptance test, all tests must be valid for all tests of partitions
- Choose inputs that
 - Force the program to generate errors
 - Cases input buffers to overflow
 - Force invalid outputs
 - Force computation results to be too large or too small

Component testing

- Software is often a composite of components that are made up of several interacting objects (classes, methods, variables)
- You access the functionality of these objects through the defined component interface
- Testing composite components should therefore focus on showing that the interfaces between the components behaves according to its specification.

Component testing guidelines

- Design tests so that parameters to a called component are at the extreme ends of their ranges
- Always test pointer parameters with null pointers.
- Design tests which cause the component to fail.
- In shared memory systems, vary the order in which components are activated.

System testing

- System testing during development involves integrating components to create a version of the system and then testing the integrated system.
- The focus in system testing is testing the interactions between components.
- System testing checks that components are compatible, interact correctly and transfer the right data at the right time across their interfaces.
- System testing tests the emergent behaviour of a system.

Release testing

- A form of system testing
- Testing a particular release of a system that is intended for use outside of the development team.
- It is used to convince the customers of the system that it is good enough for use.
 - Show that the system delivers its specified functionality, performance and dependability, and that it does not fail during normal use.
- Release testing is usually a black-box testing process
 - tests are only derived from the system specification.

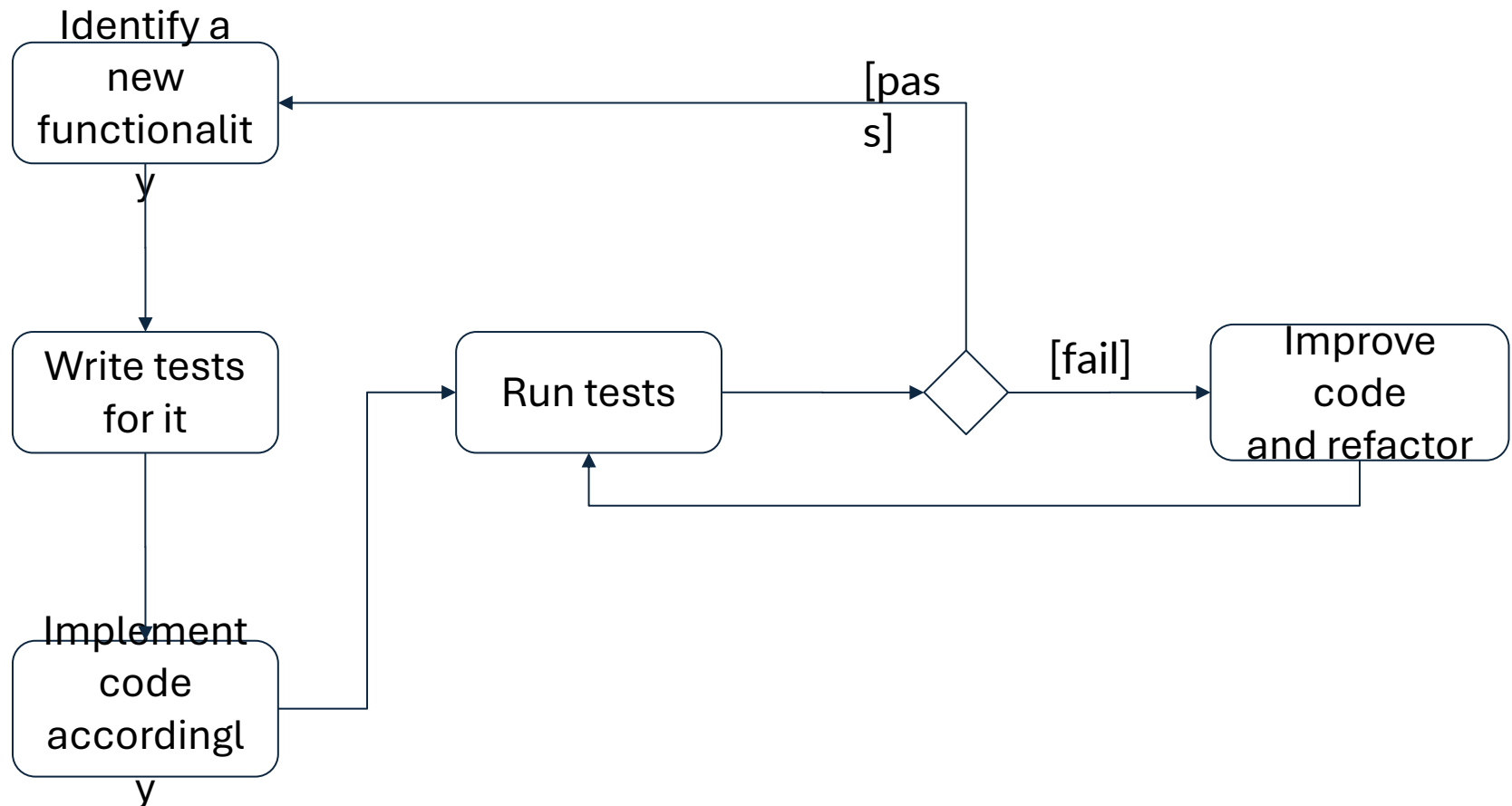
Regression testing

- Test the system to check that any changes have not ‘broken’ previously working code.
- Ideally, it requires automated testing and simple rerun is suffice
 - In a manual testing process, regression testing is expensive
- Tests must run ‘successfully’ before the change is committed.

Test-driven development

- An approach to software development in which you interleave (merge) testing and code development
- Tests are written before code
- Thus, tests and code are developed incrementally
 - You don't move to next increment until the code passes its pre-developed tests
- TDD is essential element in agile methodologies, but it can be used also in plan-driven development

Test-driven development (cont.)



TDD benefits

- Code coverage
 - Every code segment that you write has at least one associated test so all code written has at least one test.
- Regression testing
 - A regression test suite is developed incrementally as a program is developed.
- Simplified debugging
 - When a test fails, it should be easy to know where the problem.
- System documentation.
 - Tests themselves are a form of documentation that describe what the code should be doing.

User testing

- Users or customers provide input and advice on system testing.
- User testing is essential, even when comprehensive system and release testing have been carried out.
- The reason for this is that influences from the user's working environment have a major effect on the
 - Reliability
 - Performance
 - Usability
 - Robustness

User testing (cont.)

- Types of user testing
 - Alpha testing
 - Clients and development team test software at development stage
 - Beta testing
 - A release of software is made available to clients for experiment purposes
 - Acceptance testing
 - Clients test the system to decide if it is ready and be accepted